

nestjs-outbox: guia completo

Como o sistema funciona de ponta a ponta, transação por transação, e onde está cada peça no código NestJS. Estado do projeto em 06/10/2026.

Stack: NestJS 11 · PostgreSQL 17 (Drizzle ORM) · Apache Pulsar 4.1 · Zod · Jest. **Padrões:** Transactional Outbox, Inbox (consumidor idempotente), Idempotency-Key, Message Relay com `SKIP LOCKED`, webhooks assinados (Standard Webhooks), integração com API externa por Idempotency-Key, testes de falha.

Sumário

1. Visão geral e arquitetura

2. O fluxo completo em 11 passos

3. As 6 transações, uma a uma

4. Estados de cada entidade

5. Idempotência e duplicidades

6. Mensageria com Pulsar

7. Webhooks (enviados e recebidos)
8. Falhas e recuperação

9. Referência da API

10. Banco de dados

11. **Onde está cada coisa no NestJS** (mapa do código)

12. Configuração, testes e painel

13. Apêndice: exemplos de JSON

Como usar este guia

Cada seção termina com uma caixa "**Onde está no código**" com `arquivo:linha`. A Parte 11 junta tudo num mapa só, com um roteiro de leitura que segue uma requisição do começo ao fim. As linhas foram conferidas na data acima; se o código mudar, procure pelo nome da função citado ao lado.

1 Visão geral e arquitetura

O sistema é uma API de pedidos. Um cliente cria um pedido; o pagamento é confirmado; o pedido é enviado a uma transportadora (uma API externa); a transportadora avisa a entrega por webhook; e o cliente final é avisado por webhooks em cada marco, terminando com um relatório do que aconteceu.

O objetivo de engenharia é: **nenhum evento se perde, nenhum efeito é aplicado duas vezes**, mesmo com broker fora do ar, processo caindo no meio, mensagens duplicadas, API externa instável ou cliente fora do ar.

Componentes



Peça	Papel	Roda onde
API (OrdersController)	Recebe o pedido, valida, grava pedido + evento numa transação	Requisição HTTP
Relay (OutboxRelayService)	Lê a outbox e publica no broker, com retry e backoff	setInterval de 1s
Pulsar	Transporta os eventos: tópicos order.created e order.confirmed	Container outbox-pulsar
Consumidor de pedidos	Confirma o pagamento (PAID) e grava o próximo evento	Listener do Pulsar
Serviço de envio	Chama a API da transportadora com Idempotency-Key	Listener do Pulsar
Transportadora	A "outra API": cria a remessa e avisa a entrega por webhook	Simulador em /partner
Receptor de webhooks	Recebe o aviso da transportadora, valida assinatura, marca DELIVERED	Requisição HTTP
Dispatcher	Envia os webhooks para o cliente final, com retry	setInterval de 1s
Painel	Mostra tudo em tempo real e injeta falhas	public/index.html

A ideia que se repete em todo o sistema

Sempre que um passo termina, **o próximo passo é gravado na mesma transação** do banco (uma linha numa tabela de "trabalho a fazer"). Um processo separado lê essa tabela e executa, com retry. E toda operação que pode ser repetida é **idempotente**. Isso aparece 4 vezes: outbox (eventos), webhook_deliveries (avisos ao cliente), shipments (chamada externa) e inbound_webhooks (avisos recebidos).

2 O fluxo completo em 11 passos

É o mesmo fluxo da seção "Fluxo ponta a ponta" do painel (botão **Simular fluxo completo**). Tempo típico: 5 a 7 segundos, dos quais 4 são a "entrega" simulada.

#	Passo	Quem	Transação	Status depois
1	Cliente chama <code>POST /orders</code> com <code>Idempotency-Key</code>	Cliente → API	T1	pedido PENDING
2	Pedido + evento <code>OrderCreated</code> gravados juntos	API → Postgres	T1	outbox pending
3	Relay publica <code>OrderCreated</code> no Pulsar	Relay → Pulsar	T2	outbox published
4	Consumidor confirma o pedido	Pulsar → Consumidor	T3	CONFIRMED · PAID
5	Mesma transação: <code>OrderConfirmed</code> na outbox + webhook <code>order.confirmed</code>	Consumidor → Postgres	T3	
6	Relay publica <code>OrderConfirmed</code>	Relay → Pulsar	T2	
7	Serviço de envio chama a transportadora → 202	Envio → API externa	T4	SHIPPED
8	Transportadora faz a entrega	Transportadora	(do lado de lá)	
9	Webhook <code>shipment.delivered</code> chega assinado e é validado	Transportadora → API	T5	DELIVERED
10	Cliente recebe <code>order.delivered</code>	API → Cliente	T6	
11	Relatório final + webhook <code>order.completed</code>	API → Cliente	T5 + T6	

O que o cliente vê em cada momento

Momento	Resposta / aviso	status	paymentStatus	deliveryStatus
Logo após o POST	Resposta 201 do <code>POST /orders</code>	PENDING	PENDING	PENDING
~1s depois	Webhook <code>order.confirmed</code>	CONFIRMED	PAID	PENDING
~1,5s depois	<code>GET /orders/:id</code>	SHIPPED	PAID	SHIPPED
~5s depois	Webhook <code>order.delivered</code>	DELIVERED	PAID	DELIVERED
~5s depois	Webhook <code>order.completed</code> (relatório)	DELIVERED	PAID	DELIVERED

Por que o POST nunca responde PAID

A confirmação é assíncrona (outbox → Pulsar → consumidor). Fazer a API esperar jogaria fora a vantagem do outbox (responder rápido mesmo com o broker fora). O PAID chega por **webhook** (push) ou por **consulta** (`GET /orders/:id`), como a pacs.002 com ACSC no Pix.

3 As 6 transações, uma a uma

Uma "transação" aqui é um `BEGIN ... COMMIT` no Postgres: ou tudo que está dentro fica gravado, ou nada. É a ferramenta central do sistema.

T1 Criar o pedido API · síncrona

Disparo: `POST /orders` com corpo `{ customerEmail, amount }` e header `Idempotency-Key`.

1. **Validação** (antes da transação): Zod valida o corpo; `amount` é string decimal (nunca float) e vira centavos em `bigint`. Sem a chave → 400.
2. **BEGIN.**
3. `INSERT INTO idempotency_keys (key, request_hash, status='IN_PROGRESS') ON CONFLICT DO NOTHING`. A chave primária **serializa** requisições simultâneas com a mesma chave: a segunda espera a primeira terminar.
4. Se a chave já existia: mesmo corpo → **duplicata** (conta em `duplicate_requests`, responde 409 depois do commit); corpo diferente → 422.
5. `INSERT INTO orders (status PENDING).`
6. `INSERT INTO outbox (OrderCreated, tópico order.created, status pending).`
7. `UPDATE idempotency_keys → COMPLETED` com a resposta salva.
8. **COMMIT** → responde 201 com `paymentStatus: PENDING`.

Se falhar no meio: rollback de tudo. Não sobra pedido sem evento nem evento sem pedido (o botão "Testar atomicidade" prova isso).

Onde está no código

```
src/orders/orders.controller.ts:25 create() · header obrigatório na linha 32
src/orders/dto/create-order.dto.ts validação Zod · src/common/money.ts:11 toCents()
src/orders/orders.service.ts:79 create() · BEGIN na 93 · idempotency.begin 94 · duplicata 111 · INSERT orders 123 · outbox.add 145 ·
complete 162 · 409 na 172
src/idempotency/idempotency.service.ts:28 begin() · ON CONFLICT 36 · 422 na 46 · 409 "em andamento" na 51 · recordDuplicate() 63
src/outbox/outbox.service.ts:20 add() (exige um tx aberto: o evento sempre nasce numa transação)
```

T2 Relay: publicar a outbox assíncrona · a cada 1s

Três passos, com o envio **fora** de qualquer transação (nunca segurar lock esperando a rede):

1. **CLAIM** (transação curta)
`UPDATE outbox SET status='processing', attempts=attempts+1, locked_until = now() + 15s -- lease`
`WHERE id IN (SELECT id FROM outbox WHERE (status='pending' AND available_at <= now()) OR (status='processing' AND locked_until < now()) -- lease vencido ORDER BY created_at LIMIT 50 FOR UPDATE SKIP LOCKED)` -- vários relays sem conflito
2. **PUBLISH** `producer.send(...)` com `key = id` do pedido e propriedade `eventId`
3. **MARK** `sucesso → published · erro → pending + available_at = agora + backoff`
· tentativas esgotadas (8) → `failed (dead letter)`

Backoff: 1s, 2s, 4s, 8s... até 5 min, + até 1s aleatório (jitter). **Queda entre 2 e 3:** a linha fica `processing` até o lease vencer; outro ciclo publica de novo; o consumidor descarta a cópia.

Onde está no código

```
src/outbox/outbox-relay.service.ts:76 onApplicationBootstrap() (setInterval na 78) · tick() 98 · claim() 129 (SQL com SKIP LOCKED na 141) · publishOne() 152 (publish 178, queda simulada 196, MARK published 216) · markFailure() 226 (backoff 229, failed/pending 233)
src/outbox/backoff.ts:5 backoffMs() · src/outbox/outbox-cleanup.service.ts:18 expurgo diário às 03:00 (@Cron)
```

T3 Consumidor confirma o pagamento Pulsar order.created

1. **BEGIN.**
2. `INSERT INTO processed_events (event_id, 'order-confirmation') ON CONFLICT DO NOTHING`. Se não inseriu: **duplicata** → conta em `duplicates`, `commit`, `ACK`, fim.
3. `UPDATE orders SET status='CONFIRMED', confirmed_at=now() WHERE status='PENDING'` → isso é o **PAID**.
4. `INSERT INTO outbox OrderConfirmed` (tópico `order.confirmed`): o próximo salto.
5. `INSERT INTO webhook_deliveries order.confirmed` para cada endpoint que assina o evento.
6. **COMMIT** → `ACK` no Pulsar.

Se lançar erro (inclusive a falha simulada): `rollback`, inclusive do registro em `processed_events`, `NACK`, e o Pulsar reentrega em ~1s. Depois de 10 reentregas, a mensagem vai para a DLQ.

Onde está no código

```
src/consumers/order-created.consumer.ts:50 assina order.created (subscription order-confirmation) · handle() 60 · INSERT processed_events 81 · conta duplicata 90 · falha simulada 117 · UPDATE CONFIRMED 122 · outbox.add OrderConfirmed 134 · webhooks.enqueue 143
```

T4 Serviço de envio chama a transportadora Pulsar order.confirmed · chamada HTTP externa

A chamada HTTP **não cabe numa transação** do Postgres. A segurança vem de três peças:

1. Remessa já aceita para o pedido? → duplicata (conta em `processed_events.duplicates`), `ACK`, fim.
2. `INSERT INTO shipments ... ON CONFLICT (order_id) DO UPDATE attempts = attempts + 1` (status `REQUESTED`, guarda o corpo enviado).
3. `POST {CARRIER_API_URL}/shipments` com **Idempotency-Key = id do evento** e `callbackUrl` = nossa rota de webhook.
4. Falha (5xx, timeout): grava o erro na remessa e **lança** → `NACK` → o Pulsar reentrega → a chamada se repete **com a mesma chave**: a transportadora devolve a mesma remessa, nunca cria outra.
5. Sucesso (202): **BEGIN** · `processed_events · shipments ACCEPTED` (id e rastreio da transportadora) · `orders SHIPPED` · **COMMIT**.

Onde está no código

```
src/shipping/shipping.consumer.ts:41 assina order.confirmed (subscription shipping) · handle() 51 · duplicata 60 · upsert 95 · chamada 108 · NACK 127 · transação de sucesso 131 · SHIPPED 150  
src/shipping/carrier.client.ts:54 createShipment() · header Idempotency-Key na 65 · callbackUrl na 45
```

T5 Webhook de entrega chega da transportadora POST /webhooks/inbound/carrier

1. **Assinatura**: HMAC-SHA256 sobre o **corpo cru** com o segredo combinado (`CARRIER_WEBHOOK_SECRET`), janela de 5 min. Inválida → **401**.
2. **BEGIN**. `INSERT INTO inbound_webhooks (source, external_event_id, ...)` `ON CONFLICT DO NOTHING`. Mesmo id já recebido → conta em `duplicates`, **responde 200** sem reprocessar (responder erro faria o parceiro reenviar para sempre).
3. `UPDATE shipments SET DELIVERED WHERE external_id = ... AND status <> 'DELIVERED'`. Se nada mudou mas a remessa existe → evento **novo** para remessa já entregue → `200 alreadyDelivered`, nenhum aviso novo.
4. `UPDATE orders SET DELIVERED`.
5. Webhook `order.delivered` para o cliente.
6. Relatório final (`buildOrderReport`, lido nesta mesma transação) → webhook `order.completed`, com **id fixo por pedido**.
7. **COMMIT** → `200`.

Onde está no código

```
src/shipping/inbound-webhooks.controller.ts:68 rota · verify 80 · 401 na 99 · INSERT inbound 104 · alreadyDelivered 148 · order.delivered 168 · relatório 186  
src/orders/order-report.ts:69 buildOrderReport() · src/webhooks/webhooks.service.ts:40 completedEventId()  
src/main.ts:20 rawBody: true (sem isso não dá para validar a assinatura)
```

T6 Dispatcher entrega os webhooks ao cliente assíncrona · a cada 1s

Mesmo desenho do relay: CLAIM com SKIP LOCKED e lease → POST assinado (em paralelo dentro do lote, timeout 5s, sem seguir redirect) → grava a tentativa em `webhook_attempts` → 2xx = delivered; senão pending com backoff; após 6 tentativas = failed (reenvio manual).

Onde está no código

```
src/webhooks/webhook-dispatcher.service.ts:94  tick() · claim() 123 (SKIP LOCKED 139) · deliver() 146 (assinatura 156, redirect:
'manual' 178, tentativa gravada 193, failed/pending 227)
src/webhooks/webhook-signature.ts:20  sign() · verify() 35 · comparação em tempo constante 62
```

4 Estados de cada entidade

Entidade	Estados	Quem muda
Pedido (<code>orders.status</code>)	PENDING → CONFIRMED → SHIPPED → DELIVERED	T1 · T3 · T4 · T5
Pagamento (<code>paymentStatus</code> , derivado)	PENDING → PAID (quando <code>confirmed_at</code> existe)	T3
Entrega (<code>deliveryStatus</code> , derivado)	PENDING → SHIPPED (<code>shipped_at</code>) → DELIVERED (<code>delivered_at</code>)	T4 · T5
Evento (<code>outbox.status</code>)	pending → processing → published · ou → pending (backoff) · ou → failed	T1 · T2
Remessa (<code>shipments.status</code>)	REQUESTED → ACCEPTED → DELIVERED	T4 · T5
Aviso ao cliente (<code>webhook_deliveries.status</code>)	pending → delivering → delivered · ou → pending (backoff) · ou → failed	T3 · T5 · T6
Chave (<code>idempotency_keys.status</code>)	IN_PROGRESS → COMPLETED	T1

Onde está no código

Enums e tabelas: `src/db/schema.ts` (orders 29, outbox 55, shipments 192, webhook_deliveries 135) · status derivados: `src/orders/order-status.ts:9` `paymentStatusOf()` e `deliveryStatusOf()` 13 · resposta da consulta: `src/orders/orders.service.ts:46` `toView()`

5 Idempotência e duplicidades

Idempotente = repetir N vezes tem o mesmo efeito que uma. Em sistemas distribuídos, repetição é inevitável (um timeout não diz se a operação aconteceu). O sistema tem 7 camadas:

#	Onde	Chave	O que acontece na repetição	Contador
1	Requisição do cliente	Idempotency-Key	409 "transação duplicada", nenhum pedido novo	<code>idempotency_keys.duplicate_requests</code>
2	Mesma chave, outro corpo	hash SHA-256 do corpo	422	—
3	Consumidor de pedidos (inbox)	(<code>event_id</code> , <code>consumer</code>)	pula o efeito, ACK	<code>processed_events.duplicates</code>
4	Chamada à transportadora	Idempotency-Key = <code>eventId</code>	a transportadora devolve a mesma remessa	<code>shipments.attempts</code>
5	Webhook recebido	(<code>source</code> , <code>external_event_id</code>)	200 sem reprocessar	<code>inbound_webhooks.duplicates</code>
6	Webhook enviado (cada endpoint)	(<code>endpoint_id</code> , <code>event_id</code>) único	não cria segunda entrega	—
7	Cliente final	header <code>webhook-id</code> , igual nos retries	o cliente deduplica	painel: "duplicado"

Os contadores alimentam o **relatório final** (passo 11), que diz em português quantas duplicidades houve e que todas foram neutralizadas: um pedido, um pagamento, uma remessa.

Detalhe importante

No caso 1, o 409 é lançado **depois do commit**: se fosse lançado dentro da transação, o rollback apagaria a contagem da duplicata (`orders.service.ts:111` conta, `:172` lança).

6 Mensageria com Pulsar

Tópicos e assinaturas

Tópico	Produzido por	Assinatura	Consumidor
<code>persistent://public/default/order.created</code>	Relay (evento da T1)	<code>order-confirmation</code> (Key_Shared)	<code>OrderCreatedConsumer</code>
<code>persistent://public/default/order.confirmed</code>	Relay (evento da T3)	<code>shipping</code> (Key_Shared)	<code>ShippingConsumer</code>
<code>...order.created.order-confirmation.DLQ</code>	Pulsar, após 10 reentregas	—	(inspeção manual)

Caminho de uma mensagem

1. **Producer** (cache por tópico, batching desligado, timeout 5s) envia com `partitionKey` = id do pedido e propriedade `eventId`.
2. O broker grava uma **entry** num **ledger** do BookKeeper e só então devolve o `messageId` (ledger, entry). Só aí o relay marca `published`.
3. A assinatura **Key_Shared** manda mensagens da mesma chave sempre ao mesmo consumidor: ordem por pedido, paralelismo entre pedidos.
4. **ACK**: o cursor avança e a mensagem sai do backlog. **NACK**: reentrega em ~1s. 10 reentregas: **DLQ**.

Broker em memória (`BROKER=memory`): mesma interface e mesma semântica (cópia por assinatura, ordem, NACK com reentrega em 1s). Usado nos testes e2e e no teste de falhas isolado.

Por que at-least-once e não exactly-once

O relay pode publicar e cair antes de marcar; o broker pode reentregar. Exactly-once ponta a ponta não existe na prática: o que existe é **at-least-once + consumidor idempotente** = efeito exatamente uma vez.

Onde está no código

Abstração: `src/broker/message-broker.ts:45` · escolha do broker: `src/broker/broker.module.ts:18` · nomes dos tópicos: `src/events/topics.ts`
Pulsar: `src/broker/pulsar.broker.ts` `producer` 108 · `publish()` 118 (`partitionKey` 131, `send` 137) · `subscribe()` 166 (`Key_Shared` 171, `NACK 1s` 173, `DLQ` 174, `listener` 178, `ACK` 204, `NACK` 218) · `inspect()` 264 (aba Pulsar)
Memória: `src/broker/in-memory.broker.ts:33` · `drain()` 107 · `idle()` 185 (usado pelos testes)
Registro de cada mensagem (aba Mensageria): `src/messaging/messaging-log.ts:36`

7 Webhooks

7.1 Enviados ao cliente final

Evento	Quando nasce	Conteúdo principal
order.confirmed	T3 (pagamento confirmado)	status: CONFIRMED , paymentStatus: PAID , paidAt , deliveryStatus: PENDING
order.delivered	T5 (entrega avisada)	status: DELIVERED , paymentStatus , paidAt , deliveryStatus , transportadora, rastreio, quem recebeu
order.completed	T5 (logo após)	Relatório: result , status, duração, duplicates , lines numeradas, summary
webhook.test	Botão "Enviar teste"	Mensagem de teste

Cadastro de endpoints

- POST /webhooks/endpoints devolve o segredo whsec_... uma vez; nas listagens ele aparece mascarado.
- Sem events , assina ["*"] : todos os eventos, inclusive os criados no futuro. Com lista fixa, um tipo novo não chega até o endpoint assinar.
- A mesma URL não pode ter dois endpoints ativos (409 com o id existente), senão o cliente recebe tudo em dobro.
- Resultado final perdido (endpoint não assinava): POST /webhooks/orders/:id/completed reenvia só para quem ainda não recebeu (id fixo por pedido).

Assinatura (padrão Standard Webhooks)

webhook-id:	<id da entrega>	igual em todos os retries → o cliente deduplica
webhook-timestamp:	<segundos Unix>	o cliente recusa se tiver mais de 5 minutos (replay)
webhook-signature:	v1,base64(HMAC-SHA256(segredo, id + "." + timestamp + "." + corpo))	
webhook-event:	order.confirmed	
webhook-attempt:	1	

O cliente recalcula o HMAC sobre o **corpo cru** (sem re-serializar o JSON) e compara com timingSafeEqual . O header aceita várias assinaturas separadas por espaço (para troca de segredo).

7.2 Recebidos da transportadora

Ver T5. Mesmo formato de assinatura, com o segredo combinado CARRIER_WEBHOOK_SECRET . O simulador da transportadora assina, envia e retenta até 5 vezes (1s, 2s, 4s, 8s) se a nossa API falhar.

Onde está no código

Rotas: src/webhooks/webhooks.controller.ts (endpoints 28-51, reenvio do resultado 58, entregas 64-75) · DTO: src/webhooks/dto/webhook.dto.ts

Regras: src/webhooks/webhooks.service.ts createEndpoint() 62 (URL duplicada 75) · enqueue() (INSERT 157, '*' na 161) · resendCompleted() 174 · retry() 273

Envio: src/webhooks/webhook-dispatcher.service.ts · Assinatura: src/webhooks/webhook-signature.ts

Cliente de demonstração (valida a assinatura como um cliente real): src/debug/demo-receiver.controller.ts:80

Transportadora simulada: src/debug/carrier-simulator.controller.ts (503 simulado 70, idempotência 83, entrega agendada 119, webhook de volta com retry 162)

8 Falhas e recuperação

Cada falha tem um interruptor no painel ("Injetar falhas"), um teste e2e e um cenário em `pnpm test:chaos`. Todas foram executadas e passaram.

Falha	O que acontece	Como se recupera	Prova
Broker fora do ar <code>failPublish</code>	O relay não publica; evento fica pending com erro e backoff; pedido continua PENDING	Ao voltar, o próximo ciclo publica e o pedido vira PAID. Após 8 tentativas: failed + reenvio manual	e2e "broker fora do ar"; chaos #1
Consumidor quebrado <code>failConsumer</code>	Erro no handler → rollback → NACK → reentrega a cada ~1s; pedido PENDING	Ao voltar, a próxima reentrega confirma, uma vez só	e2e "consumidor com erro"; chaos #2
Queda após publicar <code>crashAfterPublish</code>	Publica mas não marca: outbox fica processing; o pedido já foi confirmado	Lease vence → publica de novo → consumidor descarta a cópia (duplicata contada)	e2e "queda após publicar"; chaos #3
Transportadora fora <code>failCarrierApi</code>	503 → serviço de envio lança → NACK; pedido PAID sem remessa	Reentrega repete a chamada com a mesma Idempotency-Key → uma remessa só	e2e "transportadora fora do ar"; chaos #4
Cliente do webhook fora <code>failWebhook</code>	Receptor responde 503; dispatcher retenta com backoff; após 6: failed	Reenvio manual (aba Webhooks ou <code>POST ../retry</code>); cliente confirma com 200	e2e "cliente fora do ar"; chaos #5

Ao rodar `pnpm test:chaos`
Use **uma única instância** ligada ao banco e ao Pulsar: outra instância sem a falha processaria os pedidos e mascararia o teste. O README mostra como subir uma instância isolada (banco `outbox_chaos`, broker em memória).

Onde está no código
Interruptores: `src/broker/chaos.service.ts` (`beforePublish()` 41) · usados em `outbox-relay.service.ts:196` (queda), `order-created.consumer.ts:117` (consumidor), `carrier-simulator.controller.ts:70`, `demo-receiver.controller.ts:127`
Rotas: `PUT /debug/chaos` em `src/debug/debug.controller.ts:240` · script: `scripts/chaos-test.mjs` · testes: `test/outbox.e2e-spec.ts`

9 Referência da API

Rotas de negócio

Rota	O que faz	Respostas
POST /orders	Cria o pedido (header <code>Idempotency-Key</code> obrigatório)	201 · 400 · 409 duplicada · 422 corpo diferente
GET /orders/:id	Pedido com status, pagamento e entrega	200 · 404
GET /orders/:id/report	Status da transação + relatório numerado	200 · 404
POST /webhooks/endpoints	Cadastra URL (responde o segredo)	201 · 409 URL repetida
GET /webhooks/endpoints	Lista (segredo mascarado)	200
PATCH /webhooks/endpoints/:id	Altera url, descrição, eventos, ativo	200 · 404
DELETE /webhooks/endpoints/:id	Remove (e o histórico)	200 · 404
POST /webhooks/endpoints/:id/test	Envia <code>webhook.test</code>	202
POST /webhooks/orders/:id/completed	Reenvia o resultado final a quem não recebeu	202 · 409 pedido não terminou
GET /webhooks/deliveries[:id]	Entregas (e tentativas)	200
POST /webhooks/deliveries/:id/retry	Reenvia uma entrega	202
POST /webhooks/inbound/carrier	Recebe <code>shipment.delivered</code> (assinado)	200 · 401 · 404
GET /health	Postgres e broker	200

Rotas de demonstração (só com `DEBUG_PANEL=true`)

Rota	O que faz
GET /debug/overview · outbox · orders · processed-events · topics	Dados do painel
GET /debug/journeys	Os 11 passos dos últimos pedidos (seção "Fluxo ponta a ponta")
GET /debug/traces · messaging · pulsar · webhooks · carrier · logs	Requisições em tempo real, Mensageria, Pulsar, Webhooks, transportadora, logs
GET/PUT /debug/chaos	Lê/liga as falhas
POST /debug/relay/run · webhooks/run	Roda um ciclo do relay / dispatcher agora
POST /debug/outbox/:id/retry · replay	Reenfileira um evento failed · republica (simula duplicata)
POST /debug/atomicity-test	Prova de rollback
POST /debug/reset	Limpa dados (mantém endpoints)
POST /partner/shipments · GET	Transportadora simulada
POST /demo-receiver	Cliente final de demonstração

Erros têm sempre o mesmo formato: `{ statusCode, error, message, details?, path, method, timestamp }` (`src/common/http-exception.filter.ts:29`).

10 Banco de dados

Tabela	Para que serve	Colunas-chave
orders	Pedidos	amount_cents (bigint, CHECK > 0), status, confirmed_at, shipped_at, delivered_at
idempotency_keys	Chaves de idempotência da API	key (PK), request_hash, response_body, duplicate_requests
outbox	Eventos a publicar	status, attempts, available_at, locked_until, last_error; índice (status, available_at)
processed_events	Inbox dos consumidores	PK (event_id, consumer), duplicates
shipments	Remessas na transportadora	order_id único, status, external_id, attempts, request/response
inbound_webhooks	Webhooks recebidos	único (source, external_event_id), headers, payload, duplicates
webhook_endpoints	URLs do cliente	url, secret, events[] (* = todos), active
webhook_deliveries	Avisos a entregar	único (endpoint_id, event_id), status, attempts, next_attempt_at
webhook_attempts	Histórico de cada POST	headers enviados, status_code, resposta, erro, duração

Migrations (pasta drizzle/)

Arquivo	O que cria
0000_init.sql	orders, outbox, processed_events, idempotency_keys
0001_webhooks.sql	webhook_endpoints, webhook_deliveries, webhook_attempts
0002_delivery_flow.sql	shipments, inbound_webhooks, status SHIPPED/DELIVERED
0003_duplicate_counters.sql	contadores de duplicidade

Geradas a partir de `src/db/schema.ts` com `pnpm db:generate`; aplicadas com `pnpm db:migrate`. A conexão e o tipo `Tx` (transação) ficam em `src/db/db.module.ts`.

11 Onde está cada coisa no NestJS

11.1 Módulos (como o NestJS monta a aplicação)

`src/main.ts` cria a aplicação com `AppModule`, liga o logger que alimenta o painel, `rawBody` (assinaturas), `enableShutdownHooks` (fecha relay, consumidores e conexões no Ctrl+C) e serve o painel de `public/`. O `AppModule` (`src/app.module.ts`) importa, nesta ordem:

Módulo	Arquivo	Fornece	Global?
ConfigModule	<code>src/config/env.ts</code>	Variáveis validadas com Zod na subida (<code>validateEnv</code>)	sim
ScheduleModule	<code>(@nestjs/schedule)</code>	Intervals e <code>@Cron</code>	—
DbModule	<code>src/db/db.module.ts</code>	Token <code>DRIZZLE</code> (o banco), tipos <code>Database / Tx</code>	sim
BrokerModule	<code>src/broker/broker.module.ts</code>	<code>MessageBroker</code> (Pulsar ou memória), <code>ChaosService</code>	sim
OutboxModule	<code>src/outbox/outbox.module.ts</code>	<code>OutboxService</code> , <code>OutboxRelayService</code> , expurgo	sim
WebhooksModule	<code>src/webhooks/webhooks.module.ts</code>	<code>WebhooksService</code> , dispatcher, rotas <code>/webhooks</code>	sim
OrdersModule	<code>src/orders/orders.module.ts</code>	Rotas <code>/orders</code> , <code>OrdersService</code> , <code>IdempotencyService</code>	—
ConsumersModule	<code>src/consumers/consumers.module.ts</code>	<code>OrderCreatedConsumer</code>	—
ShippingModule	<code>src/shipping/shipping.module.ts</code>	<code>ShippingConsumer</code> , <code>CarrierClient</code> , rota de webhook recebido	—
DebugModule (condicional)	<code>src/debug/debug.module.ts</code>	Painel, simuladores, jornada; middleware de trace em <code>POST /orders</code>	só com <code>DEBUG_PANEL</code>

Filtro global de erros (`AllExceptionsFilter`) e pipe global de validação (`ZodValidationPipe`) são registrados no `AppModule` via `APP_FILTER` e `APP_PIPE`.

11.2 Mapa por pasta

Arquivo	Responsabilidade	Principais pontos (linha)
<code>src/main.ts</code>	Bootstrap	<code>rawBody</code> 20 · <code>shutdown hooks</code> 24
<code>src/app.module.ts</code>	Composição dos módulos	<code>imports</code> · <code>APP_FILTER</code> · <code>APP_PIPE</code>
<code>src/config/env.ts</code>	Variáveis de ambiente	<code>schema</code> 8 · <code>validateEnv</code> 60
<code>src/db/schema.ts</code>	Tabelas e enums (Drizzle)	<code>orders</code> 29 · <code>outbox</code> 55 · <code>processed_events</code> 81 · <code>idempotency_keys</code> 95 · <code>webhook_endpoints</code> 110 · <code>webhook_deliveries</code> 135 · <code>webhook_attempts</code> 165 · <code>shipments</code> 192 · <code>inbound_webhooks</code> 216
<code>src/db/db.module.ts</code>	Conexão Postgres, tipos Tx	<code>createDb</code> 16
<code>src/events/topics.ts</code>	Nomes de tópicos, eventos, contratos dos webhooks, tipos de status	<code>Topics</code> · <code>EventTypes</code> · <code>WebhookEvents</code> · <code>ALL_WEBHOOK_EVENTS</code>
<code>src/orders/orders.controller.ts</code>	Rotas <code>/orders</code>	<code>POST</code> 25 · <code>GET</code> 40 · <code>report</code> 46
<code>src/orders/orders.service.ts</code>	T1: criar pedido	<code>toView</code> 46 · <code>create</code> 79 · <code>BEGIN</code> 93 · <code>report</code> 188
<code>src/orders/order-status.ts</code>	Regras PAID e <code>deliveryStatus</code>	<code>paymentStatusOf</code> 9 · <code>deliveryStatusOf</code> 13
<code>src/orders/order-report.ts</code>	Relatório numerado (passo 11)	<code>buildOrderReport</code> 69 · frases 124+
<code>src/idempotency/idempotency.service.ts</code>	Idempotency-Key	<code>begin</code> 28 · <code>recordDuplicate</code> 63 · <code>complete</code> 73
<code>src/outbox/outbox.service.ts</code>	Gravar evento (exige tx)	<code>add</code> 20
<code>src/outbox/outbox-relay.service.ts</code>	T2: relay	<code>tick</code> 98 · <code>claim</code> 129 · <code>publishOne</code> 152 · <code>markFailure</code> 226
<code>src/outbox/backoff.ts</code>	Backoff com jitter	<code>backoffMs</code> 5
<code>src/outbox/outbox-cleanup.service.ts</code>	Expurgo diário	<code>@Cron</code> 18
<code>src/broker/message-broker.ts</code>	Interface do broker	<code>MessageBroker</code> 45
<code>src/broker/pulsar.broker.ts</code>	Pulsar real	<code>publish</code> 118 · <code>subscribe</code> 166 · <code>inspect</code> 264

src/broker/in-memory.broker.ts	Broker em memória	drain 107 · idle 185
src/broker/chaos.service.ts	Interruptores de falha	flags · beforePublish 41
src/consumers/order-created.consumer.ts	T3 : confirmar pagamento	subscribe 50 · handle 60
src/shipping/shipping.consumer.ts	T4 : chamar transportadora	subscribe 41 · handle 51
src/shipping/carrier.client.ts	Cliente HTTP da transportadora	createShipment 54
src/shipping/inbound-webhooks.controller.ts	T5 : webhook de entrega	rota 68 · verify 80
src/webhooks/webhooks.controller.ts	Rotas /webhooks	28 a 75
src/webhooks/webhooks.service.ts	Endpoints, enqueue, reenvios	completedEventId 40 · createEndpoint 62 · enqueue 157 · resendCompleted 174
src/webhooks/webhook-dispatcher.service.ts	T6 : entregar webhooks	tick 94 · claim 123 · deliver 146
src/webhooks/webhook-signature.ts	HMAC sign/verify	sign 20 · verify 35
src/common/money.ts	Dinheiro em centavos	toCents 11 · fromCents 19
src/common/http-exception.filter.ts	Formato único de erro + log	AllExceptionsFilter 29
src/health/health.controller.ts	GET /health	check 19
src/logging/log-buffer.ts	Logs do painel	LogBuffer 16 · BufferedLogger 51
src/tracing/	Requisições em tempo real	trace.middleware.ts (run 47) · trace-store.ts
src/messaging/messaging-log.ts	Aba Mensageria	MessagingLog 36
src/debug/debug.controller.ts	Rotas /debug	overview 68 · journeys 218 · chaos 240 · atomicity 296 · reset 343
src/debug/journey.service.ts	Os 11 passos do painel	latest 75 · build 84 · passo 11 em 372
src/debug/carrier-simulator.controller.ts	Transportadora simulada	CarrierSimulator 53 · callback 162 · controller 230
src/debug/demo-receiver.controller.ts	Cliente final de demonstração	receive 80
public/index.html	Painel (HTML + JS sem framework)	funções renderJourney, renderWebhooks, renderPulsar, renderMessaging, renderTraces
test/outbox.e2e-spec.ts	22 testes de integração	Postgres real + broker em memória + servidores HTTP falsos
scripts/chaos-test.mjs	Testes de falha ao vivo	6 cenários

11.3 Roteiro de leitura: siga uma requisição pelo código

✓	Leia	O que conferir
☐	orders.controller.ts:25	Header obrigatório e chamada ao service
☐	orders.service.ts:79-180	Uma transação: chave, pedido, evento, resposta; 409 depois do commit
☐	idempotency.service.ts:28-60	ON CONFLICT, hash do corpo, 409/422
☐	outbox-relay.service.ts:129-240	SKIP LOCKED, lease, publish fora da transação, backoff
☐	pulsar.broker.ts:118-230	partitionKey, Key_Shared, ACK/NACK, DLQ
☐	order-created.consumer.ts:60-170	Inbox, CONFIRMED, próximo evento e webhook na mesma transação
☐	shipping.consumer.ts:51-170	Upsert, chamada com Idempotency-Key, NACK, transação de sucesso
☐	carrier-simulator.controller.ts:66-180	Como a "outra API" responde e chama de volta
☐	inbound-webhooks.controller.ts:68-210	Assinatura, dedupe, DELIVERED, order.delivered e order.completed
☐	webhook-dispatcher.service.ts:94-260	Envio assinado, tentativas, failed
☐	order-report.ts:69-220	Como as frases do relatório são montadas
☐	test/outbox.e2e-spec.ts	Cada garantia acima tem um teste com o mesmo nome

12 Configuração, testes e painel

Variáveis de ambiente (`src/config/env.ts`)

Variável	Padrão	Uso
DATABASE_URL	—	Postgres
BROKER	pulsar	pulsar ou memory
PULSAR_URL · PULSAR_ADMIN_URL	localhost:6650 · :8080	Broker e admin REST
PORT	3000	HTTP
OUTBOX_POLL_INTERVAL_MS · BATCH_SIZE · MAX_ATTEMPTS · LEASE_SECONDS · RETENTION_DAYS	1000 · 50 · 8 · 15 · 7	Relay
WEBHOOK_POLL_INTERVAL_MS · BATCH_SIZE · MAX_ATTEMPTS · TIMEOUT_MS	1000 · 20 · 6 · 5000	Dispatcher
APP_PUBLIC_URL	http://localhost:PORT	Para onde a transportadora chama de volta
CARRIER_API_URL · CARRIER_NAME	.../partner · Transportadora Simulada	A "outra API"
CARRIER_WEBHOOK_SECRET · CARRIER_TIMEOUT_MS · CARRIER_DELIVERY_DELAY_MS	whsec_carrier_demo_secret_123 · 5000 · 4000	Integração e simulador
DEBUG_PANEL	true	Painel e rotas /debug (desligar em produção)

Testes

Comando	O que roda	Resultado atual
pnpm test	Unitários: dinheiro, backoff, broker em memória, assinatura HMAC	13 passaram
pnpm test:e2e	Integração: banco outbox_test real, broker em memória, transportadora e cliente falsos	22 passaram
pnpm test:chaos	As 5 falhas ao vivo + caminho feliz, contra uma instância rodando	6 de 6 passaram

Como subir

docker compose up -d	# Postgres 17 + Pulsar 4.1
pnpm install	
pnpm db:migrate	# aplica as 4 migrations
pnpm start:dev	# API + painel em http://localhost:3000

O painel (`http://localhost:3000`)

Seção	O que mostra
Faixa superior	API → Outbox → Broker → Consumidor com contadores ao vivo
Fluxo ponta a ponta	Os 11 passos do pedido escolhido; botão Simular; passo 11 com status (pedido, pagamento, entrega, webhook final), relatório e correção quando o webhook final não sai
Novo pedido	Formulário e botões: envio duplo (409), chave reutilizada (422), sem chave, dados inválidos, 10 pedidos
Webhook do cliente final	Cadastro de endpoint ("todos os eventos" por padrão)
Injetar falhas	Os 5 interruptores
Abas	Outbox · Pedidos (com pagamento e entrega) · Eventos processados · Tópicos · Mensageria · Pulsar · Webhooks

Logs da aplicação	Logs ao vivo, erros com a resposta enviada
Requisições em tempo real	Linha do tempo de cada POST /orders

13 Apêndice: exemplos de JSON

Resposta do POST /orders (201)

```
{ "id": "103e234f-...", "customerEmail": "cliente@loja.com", "amount": "99.90", "amountCents": "9990",
  "status": "PENDING", "paymentStatus": "PENDING", "paidAt": null,
  "deliveryStatus": "PENDING", "shippedAt": null, "deliveredAt": null,
  "createdAt": "2026-10-06T17:17:49.140Z", "confirmedAt": null }
```

Duplicata (409)

```
{ "statusCode": 409, "error": "CONFLICT",
  "message": "Transação duplicada: a Idempotency-Key c5ad94a5-... já foi usada para criar um pedido",
  "details": { "idempotencyKey": "c5ad94a5-...", "originalOrderId": "a1439a5b-..." },
  "path": "/orders", "method": "POST", "timestamp": "..." }
```

Webhook order.confirmed

```
{ "id": "41dc3e4d-...", "type": "order.confirmed", "createdAt": "...",
  "data": { "orderId": "103e234f-...", "status": "CONFIRMED", "paymentStatus": "PAID",
    "paidAt": "2026-10-06T17:17:49.809Z", "deliveryStatus": "PENDING", "amount": "99.90",
    "message": "Pedido 103e234f pago e confirmado com sucesso: transação concluída." } }
```

Webhook order.delivered

```
{ "type": "order.delivered",
  "data": { "orderId": "...", "status": "DELIVERED", "paymentStatus": "PAID", "paidAt": "...",
    "deliveryStatus": "DELIVERED", "carrier": "Transportadora Simulada",
    "trackingCode": "BR300114366SP", "deliveredAt": "...", "receivedBy": "Portaria",
    "message": "Pedido ... entregue ao destinatário." } }
```

Webhook order.completed (relatório final)

```
{ "type": "order.completed",
  "data": { "orderId": "2e4b9ec0-...", "result": "DELIVERED", "paymentStatus": "PAID", "deliveryStatus": "DELIVERED",
    "durationMs": 4800,
    "duplicates": { "requests": 1, "brokerRedeliveries": 0, "carrierWebhooks": 0, "total": 1 },
    "lines": [
      { "n": 1, "text": "Às 14:29:04 o cliente enviou POST /orders (R$ 150,00, ...) com a Idempotency-Key 80aae17e-..." },
      { "n": 2, "text": "Numa única transação, o pedido 2e4b9ec0 foi criado como PENDING e o evento OrderCreated foi gravado na outbox. ..." },
      { "n": 3, "text": "A mesma requisição chegou de novo 1 vez com a mesma chave. Foi recusada com 409 ..." },
      { "n": 4, "text": "O relay publicou OrderCreated no Pulsar em +535ms, na 1ª tentativa." },
      { "n": 5, "text": "O consumidor confirmou o pedido em +538ms: pagamento PAID." },
      { "n": 6, "text": "O serviço de envio chamou a Transportadora Simulada: remessa shp_... aceita (202) em +763ms ..." }
    ],
    { "n": 7, "text": "A transportadora entregou e avisou por webhook assinado em +4,8s (assinatura válida)."},
    { "n": 8, "text": "Resultado final: pedido DELIVERED e PAID em 4,8s. Houve 1 duplicidade, neutralizada: ..." } ],
    "summary": "Pedido 2e4b9ec0 pago e entregue em 4,8s. Houve 1 duplicidade, neutralizada: ..." } }
```

Headers de um webhook enviado

```
content-type: application/json
user-agent: nestjs-outbox-webhooks/1.0
webhook-id: 88cb082a-28d0-4409-b956-e5847b68fafd
webhook-timestamp: 1791299947
webhook-signature: v1,hDv/kYtSZbCaDr4j0uuC1zRC7NKWyt2RDPgTvmA6N6w=
webhook-event: order.confirmed
webhook-attempt: 1
```

Resumo em uma frase

Cada passo grava o próximo na mesma transação, um processo separado executa com retry e backoff, e toda repetição é absorvida por uma chave única: por isso nada se perde, nada duplica, e o cliente final sempre recebe o resultado, assinado.